TECHIN 510: Week 1 Lab Report

Taya Li | Apr 4

Component A: Staff Interview

Interview Notes

Guest: Dorothy, Program Coordinator, Student Purchasing

Direct Quotes:

- "I'm not sure if there is an easier way for students to submit their orders, for her to review and an approval process"
- "Needs all the information at once"

Surprises:

- The volume ranges wildly from 20 to 150 orders per month, depending on the cohort and project phase. There is no fixed rhythm.
- Collecting non-Amazon receipts is surprisingly complex. It requires checking multiple email accounts and sometimes getting help from the front desk team.
- The budget is also a variable, not a fixed number per team.

Current Workarounds:

- Excel spreadsheets to track all purchase data (Team #, CFO, supplier, quantity, item name, price, link, notes, approval status)
- Students submit requests in unstructured ways. Information often comes in incomplete, and Dorothy has to follow up.
- Amazon purchases are straightforward.
- Non-Amazon purchases require checking different email accounts and front desk assistance to collect receipts.
- Dorothy compiles everything and submits to the instructor for approval once a week.

What Success Looks Like (in her own words):

- "An easier way for students to submit their orders": students provide all required information in one go
- "For her to review and an approval process": a cleaner workflow for Dorothy and the instructor to review and approve requests
- "Needs all the information at once": no more chasing students for missing details

Problem Statement

"When Dorothy (Program Coordinator) needs to process student purchase requests, she currently relies on Excel spreadsheets and checks multiple email accounts for receipts, which causes delays in tracking orders and a heavy manual workload, especially for non-Amazon purchases."

Component B: Streamlit App

AI Usage Log

Interaction 1: Initial App Setup

- **Prompt:** I referenced three files from my Downloads (app.py, requirements.txt, README.md) and asked Cursor to set up the project and run `pip install -r requirements.txt` followed by `streamlit run app.py`.
- **What AI produced:** Cursor copied app.py into my project workspace, added pandas to requirements.txt, installed dependencies using the project venv, and started the Streamlit server. The app became available at localhost:8501.
- **Worked on first try?** Yes. There was a urllib3/LibreSSL warning in the terminal, but it did not affect the app.
- **What I fixed:** Nothing needed to be fixed.

Interaction 2: Problem Clarification and Prioritization

- **Prompt:** I pasted Dorothy's raw interview notes (direct quotes, pain points, current process, fields tracked in Excel) and asked Cursor in Chinese to re-clarify the problem and suggest a better approach, noting that Dorothy's priority is a cleaner submission experience for students.
- **What AI produced:** Cursor broke the problem into four layers (student side, Dorothy side, process side, business reality), identified what to build first vs. later, proposed five design principles for a "cleaner submission," and asked five clarifying questions before writing code.
- **Worked on first try?** This was not a code generation step. It was a planning/analysis step that helped me make better design decisions before iterating on the app.
- **What I fixed:** N/A (no code generated in this step).

Interaction 3: Code Explanation and Manual Modification

- **Prompt:** I selected the form validation block in app.py and asked Cursor: "Explain this code to me line by line. I am a beginner."
- **What AI produced:** Cursor explained how the validation checks each required field, collects error messages into a list, and only creates a new CSV row if no errors are found. It also explained how uuid generates a unique Request ID for each submission.

- **Worked on first try?** Yes (explanation only, no code changes).
- **What I fixed:** I then manually (without AI) added a descriptive comment block at the top of app.py and changed the submit button label from "Submit request" to "Submit purchase request" to make the action clearer.

Accessibility Baseline

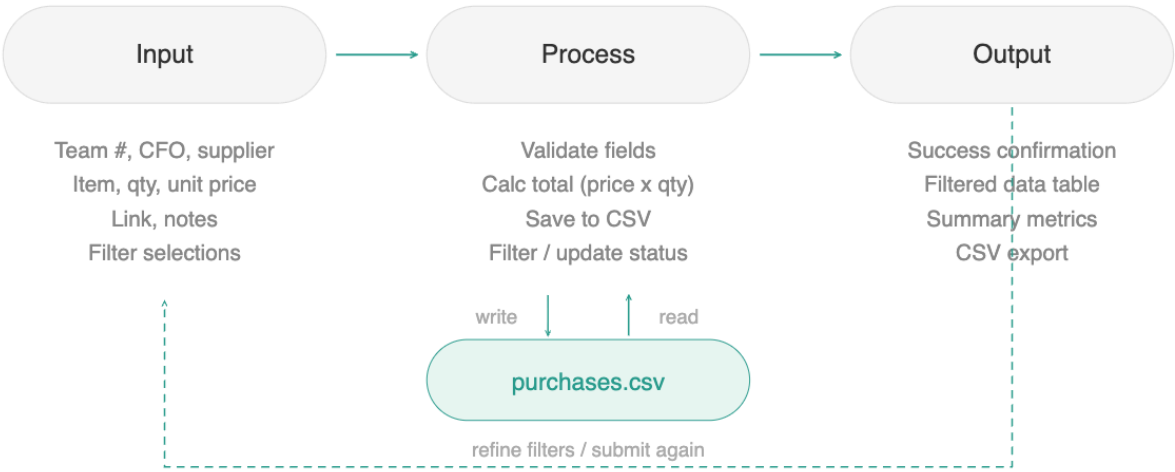
- **Color contrast check:** Pass. Streamlit's default theme uses dark text on a white background with sufficient contrast ratio.
- **Semantic headings check:** Pass. The app uses st.title for the main heading, st.header for page-level sections, and st.subheader for subsections, providing a clear heading hierarchy.

Reflection

1. What surprised you about AI-assisted coding? It was way faster than I expected. I just described what I wanted in one paragraph and Cursor gave me a working app with two pages. When I changed the quantity and the total updated by itself, that was a nice surprise.
2. What did the AI get wrong? First version used st.form, so the line total did not update when I changed the number. I had to tell Cursor to remove the form wrapper. Also the dashboard was too complicated at first, I asked it to make it simpler.
3. Could you explain your code? Yes. I asked Cursor to explain the validation part line by line. Basically it makes a list of errors, checks each field one by one, and only saves to CSV if there are no errors. It uses pandas for the CSV and uuid to make each request ID unique.
4. What did you learn from the interview? Dorothy does not need a fancy dashboard. She just wants students to give her all the info at once so she does not have to chase people. So I skipped the extra stuff like budget tracking and receipt uploads, and just made sure the form collects everything she needs in one shot.

Component C: System Architecture & Design

I/P/O Architecture Diagram



Design Decision Log

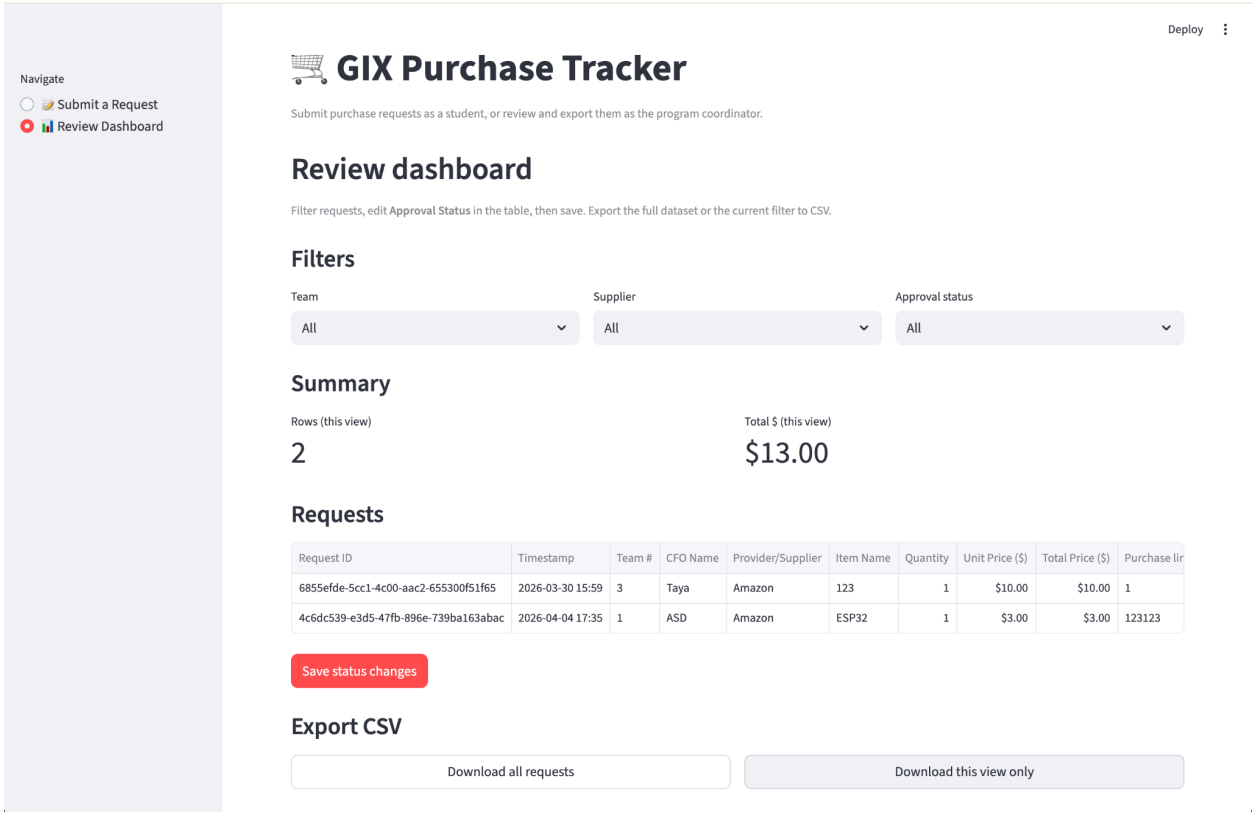
Field	Your Entry
Decision	Put everything in one file (app.py) with two pages via sidebar navigation
Alternatives considered	Split into multiple files (e.g., separate student form and admin dashboard), or use Streamlit's multi-page app structure with a pages/ folder
Why you chose this	The app is under 200 lines and has only two views. A single file keeps it simple to read, run, and debug at this stage. Cursor also generated everything in one file by default.
Trade-off	If more features are added (e.g., receipt upload, budget tracking, user authentication), the file will become hard to navigate and maintain
When would you choose differently?	If the app grew beyond 3 pages or 400+ lines, splitting into a pages/ folder with shared utility modules would be cleaner

Component D: Testing & Validation

Smoke Test Table

#	Feature Tested	Action You Took	Expected Result	Actual Result	Pass/Fail
1	Submit a purchase request	Filled in all fields (Team 3, test CFO, Amazon, ESP32 board, qty 2, $15.99, link) and clicked "Submit purchase request"	Success message showing item name, quantity, and total ($31.98)	Success message displayed correctly, row added to purchases.csv	Pass
2	Filter by team on dashboard	Switched to Review Dashboard, selected a team from the Team filter dropdown	Only requests from that team shown in the table	Table filtered correctly, summary metrics updated to match	Pass
3	Export CSV	Clicked "Download all requests" button on the dashboard	Browser downloads a CSV file containing all purchase requests	CSV file downloaded with all rows and correct column headers	Pass

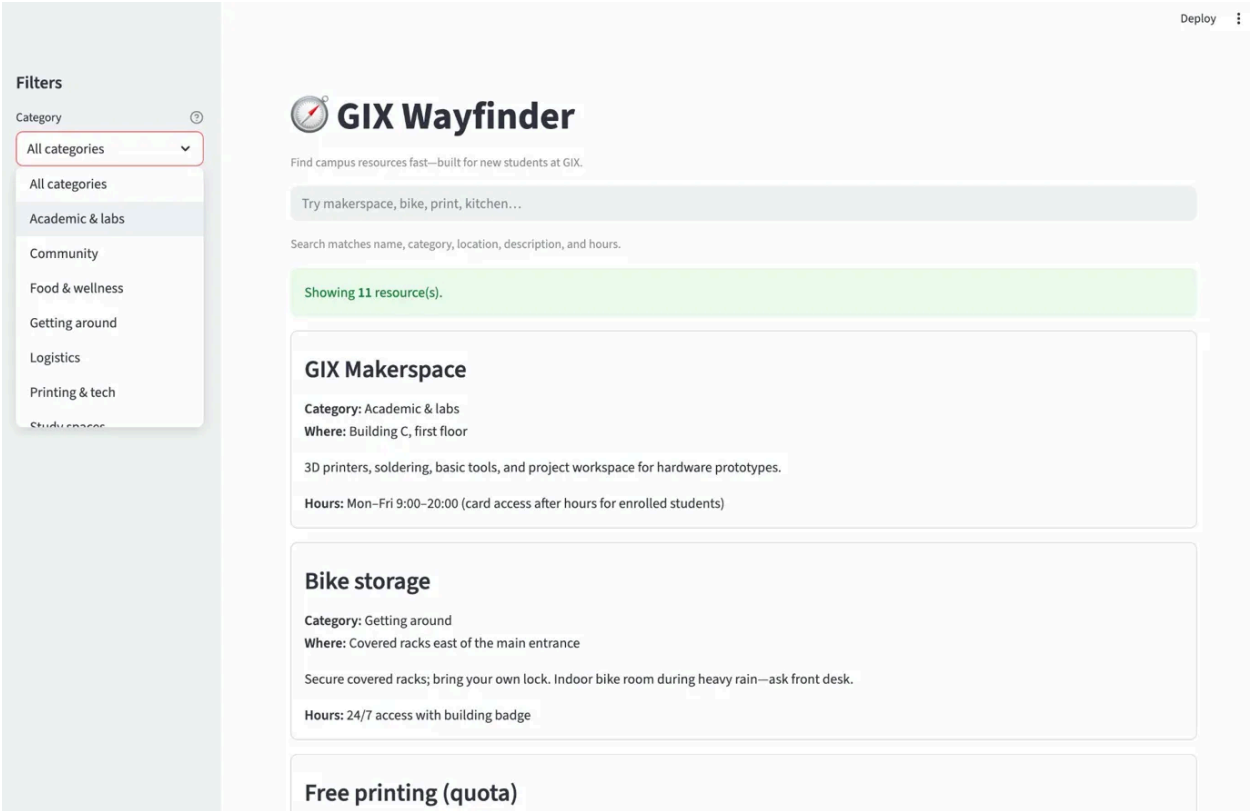
Screenshot



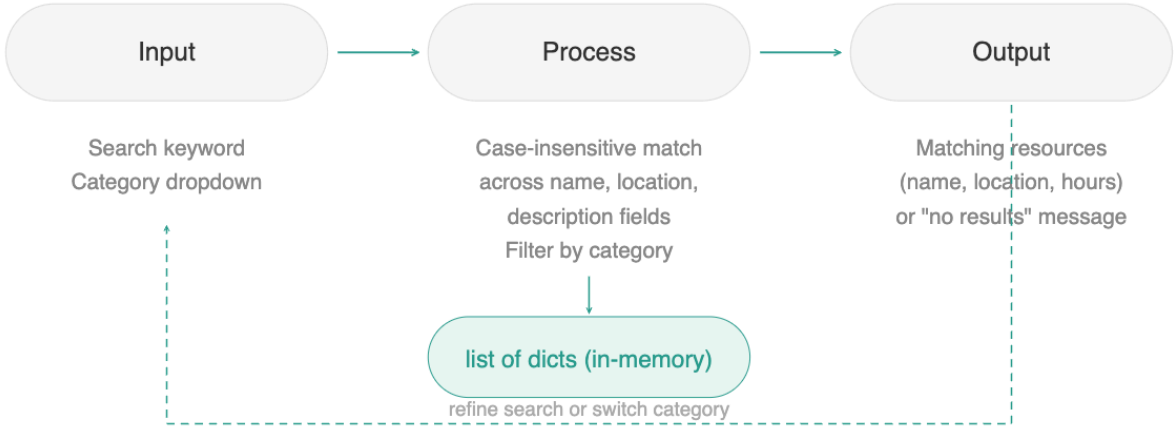
Quality Gate Checklist

- [x] Smoke test table completed with 3 tested features
- [x] Any failed test is either fixed and re-tested, or clearly documented
- [x] Screenshot of the running app included
- [x] Accessibility baseline results recorded

Component E: GIX Wayfinder App



I/P/O Diagram



Edge Case 1

- What I tested: Searched for something that does not exist on campus (e.g., "swimming pool")
- Why it matters: Without handling this, the user would see an empty page with no idea what to do next.

- Result: Pass. The app showed a friendly "no results" message with suggestions.

Edge Case 2

- What I tested: Typed in all caps (e.g., "MAKERSPACE" instead of "makerspace")
- Why it matters: If the search is case-sensitive, most users will not find what they are looking for.
- Result: Pass. Search worked fine regardless of capitalization.

Assert Statement: In wayfinder.py (lines 87-90), there is an assert that checks every resource has a non-empty name and category before the app starts. If any entry is missing these fields, the app will not run.

```
assert all(

    r.get("name") and str(r.get("name", "")).strip() and
r.get("category") and str(r.get("category", "")).strip()

    for r in RESOURCES

), "Every resource must have a non-empty name and category."
```

Prompt Log

Initial prompt: "Build a Streamlit app called 'GIX Wayfinder' that helps new students find campus resources at GIX. Store resources in a Python list of dictionaries with fields: name, category, location, description, and hours. Include at least 10 resources. Add a search bar that filters by keyword and a sidebar dropdown to filter by category. Show a friendly message when no results match. Add one assert statement to verify data integrity."

Refined prompt: "Make the search case-insensitive and also match against the description and location fields, not just the name."

What changed and why: The first version only searched by name. So if I typed "floor 2" nothing showed up even though some resources mention their floor in the description. After refining, the search checks name, description, and location together, which makes it actually useful.